Strategy Design Pattern

Intent

Define a family of interchangeable algorithms, put each one in its own class behind a common interface, and make them **swappable at runtime**. The object that uses the algorithm (the *context*) holds a reference to the interface, so you can change *what it does* by giving it a different strategy — without touching the context's code.

Here the context is a `Calculator`, and the interchangeable algorithms are the arithmetic operations: add, subtract, multiply, divide. The calculator doesn't know *how* to add or divide — it just runs whatever `OperationStrategy` it was handed.

UML class diagram

```
      <<interface>> OperationStrategy
      | +doOperation(int,int) : int |
        ^        ^        ^        ^
        |        |        |        |
   Addition  Substraction Multiplication Division
        \        |        |        /
         interchangeable algorithms
        +--------------------------+
        |        Calculator         |  (context)
        | -operationStrategy        |
        | +calculate(x,y) ----------+--> strategy.doOperation(x,y)
        +--------------------------+
```

The players

```
strategy/OperationStrategy                    the strategy interface: doOperation(x, y)
strategy/concretStrategy/AdditionOperationStrategy         x + y
                        /SubstractionOperationStrategy      x - y
                        /MultiplicationOperationStrategy     x * y
                        /DivisionOperationStrategy          x / y
context/Calculator                            the context: holds a strategy, delegates to it
DesignPatternsApplication                     the demo — builds a Calculator with a strategy
```

The code, line by line

`OperationStrategy` — the strategy interface

```
public interface OperationStrategy {
        public int doOperation(int x, int y);
}
```

- This single method **is** the contract every algorithm must satisfy: "take two ints, return an int." It's the abstraction the context depends on.
- Because everything is coded against this interface (not against `AdditionOperationStrategy` etc.), any operation can stand in for any other — that interchangeability is the whole pattern.

The concrete strategies

```
public class AdditionOperationStrategy implements OperationStrategy {
        @Override public int doOperation(int x, int y) { return x + y; }
}
public class SubstractionOperationStrategy implements OperationStrategy {
        @Override public int doOperation(int x, int y) { return x - y; }
}
public class MultiplicationOperationStrategy implements OperationStrategy {
        @Override public int doOperation(int x, int y) { return x * y; }
}
public class DivisionOperationStrategy implements OperationStrategy {
        @Override public int doOperation(int x, int y) { return x / y; }
}
```

- Each class encapsulates **exactly one algorithm** and nothing else. `AdditionOperationStrategy` knows how to add; it knows nothing about subtraction, about the calculator, or about the others.

- They all implement `doOperation` with the same signature, so from the outside they are indistinguishable — you can drop any one of them wherever an `OperationStrategy` is expected.

`Calculator` — the context

```
public class Calculator {

        private OperationStrategy operationStrategy;

        public Calculator(OperationStrategy _operationStrategy) {
                this.operationStrategy = _operationStrategy;
        }

        public int calculate(int x, int y) {
                return operationStrategy.doOperation(x, y);
        }
}
```

This is the heart of the pattern. Three parts:

- `private OperationStrategy operationStrategy;` — the context holds the strategy **by its interface type**, not by a concrete class. This is what lets it work with *any* operation. The `Calculator` has no idea whether it's holding addition or division.
- **The constructor** takes a strategy from outside and stores it. The choice of algorithm is **injected into** the context, not decided by it. (This is composition — the context *has-a* strategy.)
- `calculate(x, y)` does the actual work by **delegating** to `operationStrategy.doOperation(x, y)`. The context contributes the surrounding workflow (here, just accepting the two operands); the *variable* part — the arithmetic — is handed off to the strategy.

`DesignPatternsApplication` — the demo

```
Calculator calculator = new Calculator(new AdditionOperationStrategy());
int calculate = calculator.calculate(5, 20);
System.out.println(calculate);        // 25
```

- A `Calculator` is created with the **addition** strategy plugged in.
- `calculate(5, 20)` delegates to `AdditionOperationStrategy.doOperation(5, 20)` → `25`.
- To compute a product instead, you change **only** the strategy passed in — `new Calculator(new MultiplicationOperationStrategy())` — and `calculate(5, 20)` now returns `100`. The `Calculator` class itself is never modified.

Why the design decisions

Why Strategy instead of `if` / `else` or a `switch`?

The naive version puts every algorithm inside the context:

```
int calculate(int x, int y, String op) {
        if (op.equals("add"))      return x + y;
        else if (op.equals("sub")) return x - y;
        else if (op.equals("mul")) return x * y;
        else if (op.equals("div")) return x / y;
        ...
}
```

Problems: the context now **owns every algorithm**, the method grows every time you add an operation (violating the Open/Closed Principle — you must *modify* existing code to *extend* behavior), and the algorithms can't be reused or tested independently. Strategy replaces that conditional with **polymorphism**: each branch becomes its own class, and choosing a branch becomes choosing an object. Adding an operation means writing a **new class**, not editing the calculator.

Why does the context hold the strategy by the *interface* type?

Because that's what makes strategies interchangeable. If `Calculator` held an `AdditionOperationStrategy` field, it could only ever add. Holding an `OperationStrategy` means it accepts *any* implementation, so behavior can change without the context changing. This is the **"program to an interface, not an implementation"** principle in action.

Why inject the strategy through the constructor (composition over inheritance)?

Strategy achieves varying behavior through **composition** — the context *has* an algorithm object — rather than **inheritance**. You could instead make `AdditionCalculator`, `DivisionCalculator` subclasses, but then:

- the algorithm is fixed at compile time (you can't switch it on a live object), and
- you get a class explosion if behavior varies along more than one axis.

Injecting the strategy keeps the algorithm **swappable at runtime** and keeps each algorithm reusable outside the calculator. This is the classic *"favor composition over inheritance"* trade-off, and it's the main structural difference from Template Method (which uses inheritance).

Why one algorithm per class?

Single Responsibility: each strategy has exactly one reason to change. Division's rules (and its divide-by-zero edge case) live only in `DivisionOperationStrategy`; they can't accidentally affect addition. Each can be unit-tested in isolation.

Execution flow (the demo)

```
main
 │
 ├── new AdditionOperationStrategy()            pick the algorithm (an object)
 │
 ├── new Calculator( theStrategy )              inject it into the context
 │       └── Calculator.operationStrategy = AdditionOperationStrategy
 │
 └── calculator.calculate(5, 20)
         └── operationStrategy.doOperation(5, 20)   ← delegate to the plugged-in strategy
                └── AdditionOperationStrategy: return 5 + 20
         returns 25
 └── prints 25
```

Swap the strategy, same call, different behavior:

```
new Calculator(new MultiplicationOperationStrategy()).calculate(5, 20)  → 100
new Calculator(new SubstractionOperationStrategy()).calculate(5, 20)    → -15
new Calculator(new DivisionOperationStrategy()).calculate(20, 5)        →   4
```

The `Calculator` code is identical in every case — only the injected strategy differs.

Notes / possible extensions (not changed in the code)

- **Runtime re-selection.** The context is set up via the constructor here. Adding a setter (`setOperationStrategy(...)`) would let you switch the algorithm on an existing `Calculator` instance, not just at creation.
- **Spring-idiomatic selection.** In this project's other patterns, the strategies would typically be `@Component`s injected as a `List` (or `Map`) and chosen by an enum key — exactly like the `EnumMap` selection in the Factory module. That turns "pick a strategy" into a lookup. This module keeps it to hand-wired `new` calls so the core mechanism stays visible.
- **`SpringApplication.run(...)`** boots a context but plays no role in the pattern here — the strategy is wired manually with `new`.
- **A lambda is a strategy too.** Since `OperationStrategy` is a single-method (functional) interface, `new Calculator((x, y) -> x + y)` would work identically — a lambda is just a lightweight strategy object.

Strategy vs. its neighbors (so you don't mix them up)

- **Strategy (this module)** — pick **one** interchangeable algorithm and delegate to it; swappable at runtime via composition.
- **Template Method** — fix the algorithm's **skeleton** in a base class and let subclasses override individual steps; varies behavior via inheritance, chosen at compile time.
- **Chain of Responsibility** — pass a request along **several** handlers until one handles it; the receiver isn't chosen up front.
- **Factory Method** — chooses **which object to create**, whereas Strategy chooses **which behavior to run** on an object you already have. (They pair well: a factory can hand you the right strategy.)

Reach for Strategy when you have **multiple ways to do one thing**, you want to choose or change the approach at runtime, and you want each approach isolated, reusable, and independently testable.